Guía básica para trabajar con listas en Python

1. ¿Qué es una lista?

Una **lista** en Python es una estructura que permite guardar varios datos dentro de una sola variable.

Por ejemplo, en vez de tener esto:

fruta1 = "manzana"
fruta2 = "pera"
fruta3 = "plátano"

Podemos guardar todo en una lista:

frutas = ["manzana", "pera", "plátano"]

Una lista puede contener textos, números, valores booleanos o incluso otras listas.

numeros = [10, 20, 30, 40]
nombres = ["Ana", "Luis", "Camila"]
mezcla = ["Python", 3, True, 5.5]

---

2. Crear una lista

Para crear una lista usamos corchetes `[ ]`.

estudiantes = ["Pedro", "María", "Javiera", "Carlos"]

También podemos crear una lista vacía:

notas = []

Esto es útil cuando queremos ir agregando datos más adelante.

---

3. Acceder a elementos de una lista

Cada elemento dentro de una lista tiene una posición llamada **índice**.

En Python, los índices comienzan desde `0`.

frutas = ["manzana", "pera", "plátano"]

Elemento	Índice
manzana	0
pera	1
plátano	2

Ejemplo:

frutas = ["manzana", "pera", "plátano"]

```
print(frutas[0])
print(frutas[1])
print(frutas[2])
```

Salida:

```
manzana
pera
plátano
```

---

4. Acceder al último elemento

También podemos usar índices negativos.

frutas = ["manzana", "pera", "plátano"]

```
print(frutas[-1])
```

Salida:

plátano

El índice `-1` representa el último elemento de la lista.

print(frutas[-2])

Salida:

pera

---

5. Modificar elementos de una lista

Las listas son **modificables**, es decir, podemos cambiar sus valores.

frutas = ["manzana", "pera", "plátano"]

frutas[1] = "naranja"

print(frutas)

Salida:

["manzana", "naranja", "plátano"]

En este caso, `"pera"` fue reemplazada por `"naranja"`.

---

6. Agregar elementos a una lista

Agregar al final con `append()`

El método `append()` agrega un elemento al final de la lista.

frutas = ["manzana", "pera"]

frutas.append("plátano")

print(frutas)

Salida:

["manzana", "pera", "plátano"]

Agregar en una posición específica con `insert()`

El método `insert()` permite agregar un elemento en una posición determinada.

frutas = ["manzana", "plátano"]

frutas.insert(1, "pera")

print(frutas)

Salida:

["manzana", "pera", "plátano"]

En este caso, `"pera"` se agregó en la posición `1`.

---

7. Eliminar elementos de una lista

Eliminar por valor con `remove()`

frutas = ["manzana", "pera", "plátano"]

frutas.remove("pera")

print(frutas)

Salida:

["manzana", "plátano"]

`remove()` elimina el primer elemento que coincida con el valor indicado.

Eliminar por posición con `pop()`

frutas = ["manzana", "pera", "plátano"]

```
fruta_eliminada = frutas.pop(1)

print(frutas)
print(fruta_eliminada)
```

Salida:

```
["manzana", "plátano"]
pera
```

`pop()` elimina un elemento según su índice. Además, devuelve el elemento eliminado.

Si usamos `pop()` sin indicar índice, elimina el último elemento.

```
frutas = ["manzana", "pera", "plátano"]

frutas.pop()

print(frutas)
```

Salida:

```
["manzana", "pera"]
```

---

8. Saber cuántos elementos tiene una lista

Para conocer la cantidad de elementos usamos `len()`.

```
frutas = ["manzana", "pera", "plátano"]

cantidad = len(frutas)

print(cantidad)
```

Salida:

```
3
```

---

9. Recorrer una lista con `for`

Una de las operaciones más comunes es recorrer una lista elemento por elemento.

```
frutas = ["manzana", "pera", "plátano"]

for fruta in frutas:
    print(fruta)
```

Salida:

```
manzana
pera
plátano
```

En este ejemplo, la variable `fruta` toma uno por uno los valores de la lista.

Primero vale `"manzana"`, luego `"pera"` y finalmente `"plátano"`.

---

10. Recorrer una lista usando índices

A veces necesitamos recorrer la lista usando sus posiciones.

```
frutas = ["manzana", "pera", "plátano"]

for i in range(len(frutas)):
    print(i, frutas[i])
```

Salida:

```
0 manzana
1 pera
2 plátano
```

Aquí ocurre lo siguiente:

len(frutas)

devuelve `3`.

Entonces:

range(len(frutas))

genera los números `0`, `1` y `2`, que son los índices de la lista.

---

11. Buscar si un elemento está en una lista

Podemos usar la palabra `in`.

```
frutas = ["manzana", "pera", "plátano"]

if "pera" in frutas:
    print("La pera está en la lista")
else:
    print("La pera no está en la lista")
```

Salida:

La pera está en la lista

También podemos negar la condición usando `not in`.

```
if "sandía" not in frutas:
    print("La sandía no está en la lista")
```

---

12. Ordenar una lista

Ordenar de menor a mayor

```
numeros = [5, 2, 8, 1, 3]

numeros.sort()
```

```
print(numeros)
```

Salida:

```
[1, 2, 3, 5, 8]
```

Ordenar textos alfabéticamente

```
nombres = ["Carlos", "Ana", "Luis", "Beatriz"]

nombres.sort()

print(nombres)
```

Salida:

```
["Ana", "Beatriz", "Carlos", "Luis"]
```

Ordenar de mayor a menor

```
numeros = [5, 2, 8, 1, 3]

numeros.sort(reverse=True)

print(numeros)
```

Salida:

```
[8, 5, 3, 2, 1]
```

---

13. Obtener una parte de la lista

Podemos obtener una porción de la lista usando **slicing**.

```
numeros = [10, 20, 30, 40, 50]

print(numeros[1:4])
```

Salida:

[20, 30, 40]

Esto toma los elementos desde la posición 1 hasta antes de la posición 4.

También podemos hacer:

print(numeros[:3])

Salida:

[10, 20, 30]

O también:

print(numeros[2:])

Salida:

[30, 40, 50]

---

14. Sumar elementos de una lista numérica

Si tenemos una lista de números, podemos sumar sus elementos con `sum()`.

notas = [5.5, 6.0, 4.8, 7.0]

total = sum(notas)

print(total)

Salida:

23.3

También podemos calcular el promedio.

notas = [5.5, 6.0, 4.8, 7.0]

```
promedio = sum(notas) / len(notas)

print(promedio)
```

Salida:

5.825

---

15. Lista con datos ingresados por el usuario

Podemos pedir datos al usuario y guardarlos en una lista.

```
nombres = []

for i in range(3):
    nombre = input("Ingrese un nombre: ")
    nombres.append(nombre)

print("Lista de nombres:")
print(nombres)
```

Ejemplo de ejecución:

```
Ingrese un nombre: Ana
Ingrese un nombre: Luis
Ingrese un nombre: Pedro
Lista de nombres:
['Ana', 'Luis', 'Pedro']
```

---

16. Ejemplo práctico: guardar notas y calcular promedio

```
notas = []

cantidad = int(input("¿Cuántas notas desea ingresar?: "))

for i in range(cantidad):
```

```
    nota = float(input("Ingrese una nota: "))
    notas.append(nota)

promedio = sum(notas) / len(notas)

print("Notas ingresadas:", notas)
print("Promedio:", promedio)
```

Ejemplo de ejecución:

```
¿Cuántas notas desea ingresar?: 4
Ingrese una nota: 5.5
Ingrese una nota: 6.0
Ingrese una nota: 4.8
Ingrese una nota: 7.0
Notas ingresadas: [5.5, 6.0, 4.8, 7.0]
Promedio: 5.825
```

---

17. Ejemplo práctico: lista de compras

```
compras = []

while True:
    producto = input("Ingrese un producto o escriba 'salir': ")

    if producto == "salir":
        break

    compras.append(producto)

print("Lista de compras:")
for producto in compras:
    print("-", producto)
```

Ejemplo de ejecución:

```
Ingrese un producto o escriba 'salir': pan
Ingrese un producto o escriba 'salir': leche
Ingrese un producto o escriba 'salir': arroz
Ingrese un producto o escriba 'salir': salir

Lista de compras:
- pan
```

- leche
- arroz

---

18. Errores comunes al trabajar con listas

Error 1: Acceder a una posición que no existe

frutas = ["manzana", "pera"]

print(frutas[2])

Esto genera error porque la lista solo tiene posiciones `0` y `1`.

Correcto:

print(frutas[1])

Error 2: Olvidar que los índices comienzan en 0

nombres = ["Ana", "Luis", "Pedro"]

print(nombres[1])

Salida:

Luis

Aunque pareciera que el índice `1` es el primer elemento, en Python representa el segundo.

Error 3: Usar `remove()` con un elemento que no existe

frutas = ["manzana", "pera"]

frutas.remove("sandía")

Esto genera error porque `"sandía"` no está en la lista.

Una forma segura sería:

```
frutas = ["manzana", "pera"]

if "sandía" in frutas:
    frutas.remove("sandía")
else:
    print("La sandía no está en la lista")
```

---

19. Ejercicios propuestos

Ejercicio 1: Lista de nombres

Crea una lista con 5 nombres y muestra cada nombre usando un ciclo `for`.

Ejercicio 2: Lista de números

Crea una lista con 6 números enteros y muestra:

- La lista completa.
- El primer número.
- El último número.
- La cantidad de elementos.

Ejercicio 3: Agregar elementos

Crea una lista vacía llamada `animales`.

Luego pide al usuario ingresar 4 animales y agrégalos a la lista usando `append()`.

Ejercicio 4: Eliminar un elemento

Crea una lista de productos.

Luego pide al usuario el nombre de un producto a eliminar.

Antes de eliminarlo, verifica si existe en la lista.

Ejercicio 5: Promedio de notas

Pide al usuario ingresar 5 notas.

Guárdalas en una lista y calcula el promedio.

Ejercicio 6: Mayor y menor

Crea una lista de números y muestra:

- El número mayor.
- El número menor.

Puedes usar:

max(lista)
min(lista)

Ejercicio 7: Contar elementos mayores a 10

Crea una lista de números.

Recorre la lista y cuenta cuántos números son mayores a 10.

Ejercicio 8: Lista de compras

Crea un programa que permita ingresar productos hasta que el usuario escriba `"salir"`.

Al final, muestra todos los productos ingresados.

---

20. Resumen de funciones importantes

Función o método	¿Para qué sirve?	Ejemplo
`append()`	Agrega un elemento al final	`lista.append("dato")`
`insert()`	Agrega un elemento en una posición	`lista.insert(1, "dato")`
`remove()`	Elimina un elemento por valor	`lista.remove("dato")`
`pop()`	Elimina por índice	`lista.pop(0)`
`len()`	Cuenta los elementos	`len(lista)`
`sort()`	Ordena la lista	`lista.sort()`
`sum()`	Suma elementos numéricos	`sum(lista)`
`max()`	Obtiene el mayor valor	`max(lista)`
`min()`	Obtiene el menor valor	`min(lista)`
`in`	Verifica si un elemento existe	`"Ana" in lista`

Cierre

Las listas son una de las estructuras más importantes en Python. Permiten guardar varios datos, recorrerlos, modificarlos, eliminarlos y analizarlos.

Dominar las listas es fundamental antes de avanzar a temas como diccionarios, funciones, archivos, bases de datos o programación orientada a objetos.